

1. Introduction to Functions

◆ What is a Function?

A **function** in programming is a **block of code** that is designed to perform a specific task. Once a function is defined, it can be used multiple times within a program, which makes the code more efficient, modular, and easier to maintain.

Key features of a function:

- **Modularity:** Functions break the program into smaller, manageable parts, making complex problems easier to solve.
- **Reusability:** A function can be called multiple times without having to rewrite the same code.
- **Abstraction:** Functions help abstract complex logic, allowing the programmer to focus on the higher-level operations instead of the details.
- **Improved Readability:** Using functions with meaningful names can help improve the readability of code, making it easier to understand and debug.

Functions help avoid redundancy in a program and are a crucial part of clean and efficient coding practices.

Anatomy of a Function:

1. **Function Definition:** This is where the function is defined with a name, parameters (if any), and the code that does the work.
2. **Function Declaration (Prototype):** In some programming languages (including C), you can declare a function before defining it to tell the compiler what the function looks like.
3. **Function Call:** This is where the function is invoked from another part of the code to execute its behavior.

◆ Types of Functions

Functions can be categorized into two main types:

1. **Library Functions (Predefined Functions):**
 - These are functions that are already defined and provided by the C programming language (or any other language). You can call them directly without needing to define them.
 - These functions are part of standard libraries that come with the language, and they perform common operations.
 - **Examples:**
 - `printf()`: Used to print formatted output to the screen.
 - `scanf()`: Used to take formatted input from the user.

- **strlen()**: Returns the length of a string (excluding the null terminator).
- **sqrt()**: Calculates the square root of a number (from `math.h` library).

Example:

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n"); // Library function call
    return 0;
}
```

In this example, **printf()** is a library function that prints a message to the console.

2. User-Defined Functions:

- These are functions that the programmer creates to perform specific tasks within a program. The programmer defines both the name and the functionality of these functions.
- User-defined functions allow for customization and flexibility in solving specific problems.
- **Benefits of User-Defined Functions:**
 - **Code Reusability:** Once defined, these functions can be reused in multiple places without rewriting the same code.
 - **Easier Debugging:** Since the code is modular, it's easier to isolate and test individual parts.
 - **Code Organization:** Helps organize code into logical units, making it more readable and maintainable.

Components of a User-Defined Function:

- **Return Type:** Specifies the type of value the function will return (e.g., `int`, `void`, `char`).
- **Function Name:** The name used to refer to the function.
- **Parameters (optional):** Values that can be passed to the function when it is called. These are optional and depend on the function's purpose.
- **Function Body:** The block of code that gets executed when the function is called.

Example:

```
#include <stdio.h>

// Function definition
int add(int a, int b) {
    return a + b; // Returns the sum of a and b
}

int main() {
    int result = add(5, 3); // Function call
}
```

```
    printf("Result: %d\n", result); // Output: 8
    return 0;
}
```

In this example, `add()` is a user-defined function that takes two integers as input and returns their sum.

◆ Why Use Functions?

Using functions in programming offers several advantages:

1. **Avoiding Code Duplication:**
 - Instead of writing the same code repeatedly, you can define it once inside a function and call it wherever needed.
2. **Easier Debugging and Maintenance:**
 - If a problem occurs, it's easier to isolate the error in the specific function where the logic resides. Additionally, any updates or fixes need only be done within the function.
3. **Better Organization and Structure:**
 - Functions allow for logical structuring of your program. You can group related tasks together inside a function, making the code much easier to read and understand.
4. **Improved Code Readability:**
 - A well-named function that performs a clear task can improve the readability of the program. It abstracts away complex logic and allows others to quickly grasp the purpose of the code.

◆ Function Components (Detailed)

1. **Function Declaration (Prototype):**
 - A declaration provides the compiler with information about the function's return type, name, and parameters before its definition. This is useful for calling functions before their actual definitions.

Syntax:

```
returnType functionName(parameter1, parameter2, ...);
```

Example:

```
int add(int, int); // Function prototype for a function that adds two integers
```

2. Function Definition:

- This is where you provide the actual code for the function's behavior.

Syntax:

```
returnType functionName(parameter1, parameter2, ...) {  
    // Code block  
}
```

Example:

```
int add(int a, int b) {  
    return a + b; // Returns the sum  
}
```

3. Function Call:

- Once a function is defined, it can be called from other parts of the program. When a function is called, the program jumps to that function, executes its code, and returns the result.

Example:

```
int result = add(5, 3); // Function call
```

Key Takeaways from this Hour:

1. **Functions are Essential** for breaking down complex problems into smaller, manageable tasks.
2. **Library Functions** are pre-defined in the language and provide commonly-used operations.
3. **User-Defined Functions** allow the programmer to create functions that are tailored to specific needs, promoting modularity and code reuse.
4. Functions contribute to **code organization, maintainability, and debugging**.
5. A function has several parts: **declaration, definition, and call**.

. Components of a Function (0.5 Hour)

In C, a function typically consists of three main components: **Function Declaration, Function Definition, and Function Call**. Each of these components plays a crucial role in how the function operates within the program.

1. Function Declaration (Prototype)

Function Declaration provides the compiler with information about the function before it is used. It is often referred to as the **function prototype**. The function declaration tells the compiler the function's **return type**, **name**, and the types of **parameters** it expects.

Purpose:

- The **declaration** informs the compiler about the function's return type, name, and parameters. This is necessary for type-checking when calling the function, especially when the function is called before it is defined.

Syntax:

```
returnType functionName(parameter1Type, parameter2Type, ...);
```

- **returnType**: Specifies the type of value the function will return (e.g., int, void, float).
- **functionName**: The name of the function.
- **parameter1Type, parameter2Type, ...**: The types of parameters the function accepts (optional).

Example:

```
int add(int, int); // Function declaration (prototype) for add function
```

Here, the `add()` function is declared to accept **two int parameters** and return an `int`.

Why is it important?

- **Forward Declaration**: The declaration allows you to call the function **before** its definition.
- **Type Safety**: It helps the compiler check if the parameters passed match the expected types and ensures the correct return type is used.

Example with Complete Program:

```
#include <stdio.h>

// Function Declaration
int add(int, int); // Declaring the add function

int main() {
    int result = add(5, 3); // Function Call
    printf("Result: %d\n", result);
    return 0;
}

// Function Definition
int add(int a, int b) {
    return a + b; // Function Definition - performs addition
}
```

```
}
```

In the above example:

- The function **add()** is declared before `main()` to allow it to be called inside `main()` even though its full definition appears later.

2. Function Definition

The **Function Definition** contains the actual code that will be executed when the function is called. This includes the function's **parameters**, **return type**, and the **body** of the function that performs the desired task.

Purpose:

- The function definition **implements the logic** of what the function is supposed to do when invoked.

Syntax:

```
returnType functionName(parameter1Type parameter1, parameter2Type parameter2,
...) {
    // Code block
    // Return statement (optional)
}
```

- **Function Name:** Matches the declaration.
- **Function Parameters:** The actual parameters (values) passed to the function when called.
- **Function Body:** The block of code that defines what the function does.

Example:

```
int add(int a, int b) {
    return a + b; // Returns the sum of a and b
}
```

In the above example, the function **add()** is defined to take two integers, **a** and **b**, and return their sum.

Why is it important?

- The **definition** provides the **function's behavior**, i.e., what happens when the function is invoked. Without the definition, the function does nothing, even if it is declared and called.

3. Function Call

A **Function Call** is used to invoke the function and execute the code within the function's body. When a function is called, the program control jumps to the function definition, executes the code, and then returns to the calling location.

Purpose:

- The **function call** tells the program to **execute** the function's code.
- It allows passing **arguments** to the function and retrieving the **return value** (if any).

Syntax:

```
functionName(parameter1, parameter2, ...);
```

- **functionName**: The name of the function being called.
- **Parameters**: The values passed to the function, matching the type and number of parameters declared in the function.

Example:

```
int result = add(5, 3); // Function call: add 5 and 3
```

In this example, the function **add()** is called with the arguments **5** and **3**. The result of the function call (the sum of 5 and 3) is stored in **result**.

Why is it important?

- **Function Call** allows you to **reuse** the functionality provided by the function multiple times in the program. Without calling a function, it would never be executed.
- It helps **pass values** to functions and **get results** back.

Summary: Components of a Function

1. Function Declaration:

- Tells the compiler about the function's return type, name, and parameters before it is used in the program.
- Example:

```
int add(int, int);
```

2. Function Definition:

- Provides the actual code that performs the task. It defines how the function works.
- Example:

```
int add(int a, int b) {
    return a + b;
}
```

3. Function Call:

- Invokes the function to execute its behavior.
- Example:

```
int result = add(5, 3); // Function call to add() with
parameters 5 and 3
```

3. Function Parameters (0.5 Hour)

◆ What are Function Parameters (Arguments)?

Function **parameters** (also known as **arguments**) are used to pass values into functions. These values can be used by the function to perform specific tasks. Without parameters, a function would only have access to its own internal data, limiting its ability to be flexible and reusable. Parameters allow functions to work with data passed from other parts of the program.

When calling a function, you provide **actual values** (arguments) that correspond to the **formal parameters** defined in the function.

◆ Types of Parameters

Parameters can be classified based on how they are passed to a function:

1. **Formal Parameters:** These are the parameters declared in the function definition. They act as placeholders for the values (arguments) that will be passed to the function when it is called.
2. **Actual Parameters:** These are the values or variables passed to the function during the function call. They correspond to the formal parameters in the function definition.

◆ Syntax of Function Parameters

The syntax for defining a function with parameters involves specifying the parameter types in the function definition.

Function Declaration/Definition Syntax

```
returnType functionName(parameterType1 parameter1, parameterType2 parameter2, ...);
```

- **parameterType1, parameterType2, ...:** The data types of the parameters (e.g., `int`, `float`, `char[]`).
- **parameter1, parameter2, ...:** The names of the parameters that will be used inside the function.

Example:

```
void greet(char name[]) {  
    printf("Hello, %s!\n", name);  
}
```

Here, **`greet()`** is a function that takes a **single parameter** `name` of type `char[]` (a string or array of characters). When the function is called, it will print a greeting message with the passed name.

◆ Function Parameters in Action

1. Function Definition

A function can be defined to accept parameters that represent the input values the function will work with. These values are provided when the function is called. The values passed into the function can be variables, literals, or expressions.

Example:

```
c  
  
#include <stdio.h>  
  
// Function with a parameter to greet a person by their name  
void greet(char name[]) {  
    printf("Hello, %s!\n", name); // Access the 'name' parameter and print  
    it  
}  
  
int main() {  
    char myName[] = "Ajaya"; // Define a string  
    greet(myName); // Call function with myName as the argument  
    return 0;  
}
```

Explanation:

- **Parameter:** `char name[]` is the formal parameter of the function `greet()`.
- **Argument:** `myName` is the actual parameter (argument) passed to the function `greet()` when it is called.
- **Output:**

```
Hello, Ajaya!
```

2. Passing Multiple Parameters

Functions can accept multiple parameters, allowing them to perform operations with more than one piece of data.

Example:

```
c
#include <stdio.h>

// Function to add two numbers
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 7); // Pass 5 and 7 as arguments to the add function
    printf("The sum is: %d\n", result); // Output: The sum is: 12
    return 0;
}
```

Explanation:

- **Function Definition:** The function `add()` has two parameters, `a` and `b`, which are both integers.
- **Function Call:** When calling `add(5, 7)`, the arguments `5` and `7` are passed to the function, and the sum is returned.

Output:

```
The sum is: 12
```

◆ Types of Parameters in C

Parameters in C can be passed in two primary ways:

1. **Call by Value** (most common):

- In **call by value**, a **copy** of the actual parameter (argument) is passed to the function.
- Any modification made to the parameter inside the function does not affect the original argument in the calling function.

Example:

```
c

void increment(int a) {
    a = a + 1;  // Modify the local copy of 'a'
}

int main() {
    int x = 5;
    increment(x);  // Pass x by value
    printf("x after function call: %d\n", x);  // x remains unchanged:
5
    return 0;
}
```

Explanation:

- **Call by Value:** The function `increment()` only modifies the local copy of the parameter `a`, and the original `x` in `main()` is unaffected.

2. **Call by Reference** (using pointers):

- In **call by reference**, the **memory address** (reference) of the actual parameter is passed to the function.
- This allows the function to **modify** the original argument, as the function has access to its memory location.

Example:

```
c

void increment(int *a) {
    *a = *a + 1;  // Modify the actual value at the address pointed by
'a'
}

int main() {
    int x = 5;
    increment(&x);  // Pass x by reference using its address
    printf("x after function call: %d\n", x);  // x is now modified: 6
    return 0;
}
```

Explanation:

- **Call by Reference:** The function `increment()` modifies the original `x` by using the pointer to its memory location.

◆ Parameter Types and Passing Data

Parameters in C can be of any data type:

- **Basic Data Types:** `int`, `float`, `char`, etc.
- **Arrays:** Arrays are passed by reference (i.e., the memory address of the first element is passed).
- **Pointers:** Pointers are often used to pass large amounts of data (like arrays or structures) efficiently.
- **Structures:** You can pass entire structures as parameters to functions.

Example (Passing an array):

```
c
#include <stdio.h>

void printArray(int arr[], int size) {
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printArray(arr, 5); // Pass array and its size to the function
    return 0;
}
```

Explanation:

- The **array `arr[]`** is passed to the function by reference, allowing the function to access the array directly.

◆ Default Parameters (C does not support)

Unlike some other programming languages (like C++), C does not support **default parameters**. However, default values can be simulated by overloading functions or using conditional checks inside the function. In C, we typically rely on **function overloading** in C++ or handling default values manually within the function.

4. Library vs User-Defined Functions (0.5 Hour)

In C programming, functions can be categorized into two broad types: **Library Functions** and **User-Defined Functions**. Understanding the differences between these two helps in determining when to use each type for efficient programming.

◆ Library Functions

Library functions are predefined functions that come with the C standard library or other external libraries. These functions perform common tasks and are ready to use without needing any custom coding. They provide essential functionality, saving time and effort by offering tested, reliable code for performing frequently required operations.

Features:

- **Provided by:** C Standard Library or external libraries.
- **Customization: Not customizable.** You cannot modify the functionality of library functions, but you can use them to perform specific tasks.
- **Examples:**
 - `printf()` – Outputs formatted text to the screen.
 - `scanf()` – Reads formatted input from the user.
 - `sqrt()` – Computes the square root of a number.
 - `strlen()` – Returns the length of a string.
 - `malloc()` – Allocates memory dynamically.

Benefits:

- **Saves Time:** You don't need to write common operations like input/output or mathematical calculations from scratch.
- **Reliability:** Library functions are extensively tested and optimized for performance.
- **Standardization:** They are part of the standard C library, ensuring consistency across C programs.

Example:

```
c

#include <stdio.h>
#include <math.h>

int main() {
    double result = sqrt(25.0); // Using the library function sqrt()
    printf("Square root of 25 is: %.2f\n", result);
    return 0;
}
```

```
}
```

In this example, the `sqrt()` function is a library function that computes the square root of the number passed to it.

◆ User-Defined Functions

User-Defined Functions are functions that are created by the programmer to perform specific tasks. These functions allow for customization and flexibility, making your code more modular, reusable, and easy to maintain. The programmer has control over the logic, the function's name, and the parameters passed.

Features:

- **Provided by:** The programmer (user).
- **Customization: Fully customizable.** You can design your own function, define its logic, parameters, and return type according to your needs.
- **Examples:**
 - `add()` – A function that adds two numbers.
 - `greet()` – A function that prints a greeting message.

Benefits:

- **Customization:** You can define the function to do exactly what you need it to.
- **Modularity:** Functions help divide your code into smaller, manageable chunks, improving organization.
- **Reusability:** Once defined, a function can be reused multiple times without rewriting the same logic.

Example:

```
c
#include <stdio.h>

// User-defined function to add two integers
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3); // Calling the user-defined function 'add'
    printf("Sum of 5 and 3 is: %d\n", result);
    return 0;
}
```

In this example, the `add()` function is a user-defined function that takes two integers as parameters, adds them, and returns the result.

◆ Comparison: Library Functions vs User-Defined Functions

Feature	Library Function	User-Defined Function
Provided by	C Standard Library or External Libraries	The Programmer
Customization	✗ Not Customizable	✓ Fully Customizable
Examples	<code>printf()</code> , <code>sqrt()</code> , <code>strlen()</code>	<code>add()</code> , <code>greet()</code> , <code>multiply()</code>
Purpose	To perform common tasks (I/O, math, etc.)	To perform specific tasks defined by the programmer
Availability	Always available (as part of the standard library)	Available only after definition by the programmer
Control	No control over the function's logic	Full control over the function's behavior and implementation
Reusability	Reusable across any program	Reusable within the scope of the program once defined

◆ Key Differences Between Library Functions and User-Defined Functions

- Provided by:**
 - Library functions are already built into the C language and are available for use by default.
 - User-defined functions are written by the programmer to perform specific tasks that aren't covered by standard library functions.
- Customization:**
 - Library functions **cannot** be customized, meaning their behavior is fixed as defined by the C standard library.
 - User-defined functions, on the other hand, are **fully customizable**. You can define them to behave exactly the way you need them to.
- Examples:**
 - Library functions such as `printf()` or `sqrt()` handle common operations like output or mathematical calculations.
 - User-defined functions such as `add()` or `greet()` are typically created for specific tasks like summing two numbers or printing a greeting message.
- Control:**
 - With **library functions**, you have **no control** over how the function works internally, but you can use it to achieve certain results.

- With **user-defined functions**, you have **full control** over how the function behaves because you write the logic yourself.
5. **Availability:**
- **Library functions** are available by default when you include the appropriate header files (e.g., `#include <stdio.h>` for input/output functions or `#include <math.h>` for mathematical functions).
 - **User-defined functions** must be explicitly defined by the programmer within the program before they can be used.

◆ Why Use Library Functions?

- **Time-Saving:** Library functions save the programmer time by providing ready-made solutions for common tasks, such as handling user input/output, mathematical operations, string manipulations, and more.
- **Reliability:** These functions are extensively tested and optimized, ensuring they are reliable and bug-free.
- **Standardization:** Library functions are part of the C standard, ensuring that any program using them will behave the same way on any system that supports the standard C library.

◆ Why Use User-Defined Functions?

- **Customization:** You can define your own function to perform exactly what you need. This is particularly useful for repetitive tasks specific to your program's functionality.
- **Modularity:** Breaking a program into smaller, logically organized functions makes it more readable and easier to debug.
- **Reusability:** Once defined, user-defined functions can be reused multiple times, which reduces code duplication and enhances maintainability.

◆ Example Comparison

1. **Library Function Example** (`sqrt()` from the `math.h` library):

```
c

#include <stdio.h>
#include <math.h>

int main() {
    double number = 16;
    double result = sqrt(number); // Using library function sqrt()
    printf("Square root of %.2f is %.2f\n", number, result);
}
```



```
    return 0;
}
```

2. User-Defined Function Example (add() function to add two numbers):

```
c
#include <stdio.h>

int add(int a, int b) {
    return a + b; // Add two numbers
}

int main() {
    int result = add(5, 3); // Calling the user-defined function
    printf("Sum of 5 and 3 is: %d\n", result);
    return 0;
}
```

5. Different Forms of User-Defined Functions (1 Hour)

In C programming, **user-defined functions** can be categorized based on the number of parameters they take and whether they return a value or not. These categories allow you to create functions that suit specific tasks and design requirements.

◆ 1. No Arguments, No Return Value

This is the simplest form of a function. It **does not take any arguments** and **does not return any value**. The function's purpose is typically to perform an action such as printing a message or modifying global variables.

Syntax:

```
c
void functionName() {
    // Code block
}
```

- **Return Type:** `void` indicates that the function does not return a value.
- **Parameters:** The function does not take any parameters.

Example:

```
c
#include <stdio.h>
```

```
// Function with no arguments and no return value
void display() {
    printf("Welcome!\n");
}

int main() {
    display(); // Function call
    return 0;
}
```

Explanation:

- **display():** This function prints the message "Welcome!" to the screen.
- Since it does not take any parameters or return a value, it is categorized as a **no arguments, no return value** function.

Output:

Welcome!

◆ 2. Arguments, No Return Value

This type of function takes **arguments (parameters)** but does **not return any value**. The function may perform operations using the arguments, but it does not return any result to the caller.

Syntax:

```
c
void functionName(parameter1Type parameter1, parameter2Type parameter2, ...)
{
    // Code block
}
```

- **Return Type:** `void` (does not return anything).
- **Parameters:** One or more parameters are passed to the function when it is called.

Example:

```
c
#include <stdio.h>

// Function with arguments and no return value
void greet(char name[]) {
    printf("Hello %s!\n", name);
}
```

```
int main() {
    greet("Alice"); // Function call with argument
    return 0;
}
```

Explanation:

- **greet()**: This function accepts a string `name` and prints a greeting message.
- The function **does not return** anything, but it **uses the argument** `name` to display a personalized message.

Output:

Hello Alice!

◆ 3. No Arguments, Return Value

This type of function **does not take any arguments** but **returns a value**. The function can be used to compute or generate a result based on internal logic, without requiring external input.

Syntax:

```
c
returnType functionName() {
    // Code block
    return value;
}
```

- **Return Type:** The type of value returned by the function (e.g., `int`, `float`).
- **Parameters:** The function does not take any arguments.

Example:

```
c
#include <stdio.h>

// Function with no arguments and a return value
int getNumber() {
    return 10;
}

int main() {
    int number = getNumber(); // Function call and capturing the return
    value
    printf("Returned number: %d\n", number);
    return 0;
}
```

Explanation:

- **getNumber()**: This function **does not take any parameters** but **returns** the integer value 10.
- The returned value is captured and stored in the variable `number`, which is then printed in the `main()` function.

Output:

```
Returned number: 10
```

◆ 4. Arguments and Return Value

This form of function **takes one or more arguments** and **returns a value**. This is the most common type of function used for operations like mathematical calculations, string manipulation, etc.

Syntax:

```
c
returnType functionName(parameter1Type parameter1, parameter2Type parameter2,
...) {
    // Code block
    return value;
}
```

- **Return Type:** The type of value returned by the function.
- **Parameters:** One or more parameters that are passed when calling the function.

Example:

```
c
#include <stdio.h>

// Function with arguments and a return value
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3); // Function call with arguments
    printf("Sum of 5 and 3 is: %d\n", result);
    return 0;
}
```

Explanation:

- `add()`: This function takes two integers (a and b) and **returns their sum**.
- The function is called with arguments 5 and 3, and the result (sum) is returned and stored in `result`.

Output:

Sum of 5 and 3 is: 8

Summary of Different Forms of Functions

Type of Function	Arguments	Return Type	Description
No Arguments, No Return Value	None	void	A function that performs an action but does not take input or return output.
Arguments, No Return Value	One or more	void	A function that takes parameters, performs an action, but does not return anything.
No Arguments, Return Value	None	A type (e.g., <code>int</code> , <code>float</code>)	A function that performs an action but returns a value.
Arguments and Return Value	One or more	A type (e.g., <code>int</code> , <code>float</code>)	A function that takes parameters and returns a value.

Practical Applications

- No Arguments, No Return Value:**
 - Useful for displaying static messages, performing logging, or controlling program flow.
 - Example: Displaying a welcome message when the program starts.
- Arguments, No Return Value:**
 - Common for functions that perform operations on input values but do not need to return any result (e.g., printing values, modifying global variables).
 - Example: Printing a personalized greeting.
- No Arguments, Return Value:**
 - Useful when the function generates or computes a value internally (e.g., random number generation, constant returns).
 - Example: Returning a default value or calculated result like the number of items in a list.
- Arguments and Return Value:**
 - This is the most commonly used form. Functions like `add()`, `multiply()`, and `findMax()` take input and return calculated results.
 - Example: A function to calculate the sum or perform any mathematical operation.

// C program to use function with no argument and no return values

```
#include <stdio.h>
void sum()
{
    int x, y;
    printf("Enter x and y\n");
    scanf("%d %d", &x, &y);
    printf("Sum of %d and %d is: %d", x, y, x + y);
}
```

```
// Driver code
int main()
{
    // function call
    sum();

    return 0;
}
```

// C program to use function with no argument and with return values

```
#include <stdio.h>

int sum()
{
    int x, y, s = 0;
    printf("Enter x and y\n");

    scanf("%d %d", &x, &y);
    s = x + y;
    return s;
}
```

```
// Driver code
int main()
{
    // function call
    printf("Sum of x and y is %d", sum());
    return 0;
}
```

// C program to use function with argument and no return values

```
#include <stdio.h>

void sum(int x, int y)
{
    printf("Sum of %d and %d is: %d", x, y, x + y);
}
```

```
// Driver code
int main()
{
    int x, y;
    printf("Enter x and y\n");

    scanf("%d %d", &x, &y);
}
```

```

// function call
sum(x, y);

return 0;
}
// C program to use function with argument and with return values
#include <stdio.h>
int sum(int x, int y) { return x + y; }
// Driver code
int main()
{
    int x, y;
    printf("Enter x and y\n");
    scanf("%d %d", &x, &y);

    // function call
    printf("Sum of %d and %d is: %d", x, y, sum(x, y));
    return 0;
}

```

Recursion and Recursive Functions in C

◆ What is Recursion?

Recursion is a programming technique where a function calls itself in order to solve a problem. The idea is to break a complex problem into smaller, more manageable sub-problems of the same kind, solve those smaller problems, and combine the results.

★ Key Idea:

“A problem is solved by solving a smaller instance of the same problem.”

Structure of a Recursive Function

A recursive function must have two parts:

Base Case

The condition where the function stops calling itself.

Prevents infinite recursion.

Recursive Case

The part where the function calls itself with a smaller input.

◆ General Form:

c

```

return_type function_name(parameters) {
    if (base_condition)
        return some_value; // Base Case
    else
        return function_name(smaller_problem); // Recursive Call
}

```

How Recursion Works (with Call Stack)

Each recursive call is pushed onto the stack and execution pauses until that call returns. When the base case is reached, the calls start returning in reverse order (last in, first out).

Example 1: Factorial Function

? Problem:

Find factorial of n where

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

✓ Code:

```

#include <stdio.h>

int factorial(int n) {
    if (n == 0)
        return 1; // Base case
    else
        return n * factorial(n - 1); // Recursive call
}

int main() {
    int num = 5;
    printf("Factorial of %d is %d", num, factorial(num));
    return 0;
}

```


🔄 Execution Flow (n = 3):

factorial(3)

→ 3 * factorial(2)

→ 2 * factorial(1)

→ 1 * factorial(0)

→ return 1

← return 1

← return 2 * 1 = 2

← return 3 * 2 = 6

◆ Example 2: Fibonacci Series

? Problem:

Fibonacci Series: 0 1 1 2 3 5 8 ...

Each term = sum of previous two terms.

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

✓ Code:

```
#include <stdio.h>
```

```
int fibonacci(int n) {
```

```
    if (n == 0) return 0;
```

```
    else if (n == 1) return 1;
```

```
    else return fibonacci(n - 1) + fibonacci(n - 2);
```

```
}
```

```
int main() {
```

```
    int n = 6;
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", fibonacci(i));
```

```
    }
```

```
    return 0;
```

}

🌀 Execution Flow for fibonacci(4):

fibonacci(4)

→ fibonacci(3) + fibonacci(2)

→ (fibonacci(2) + fibonacci(1)) + (fibonacci(1) + fibonacci(0))

→ (fibonacci(1) + fibonacci(0)) + 1 + 1 + 0

→ 1 + 0 + 1 + 1 + 0 = 3

🌀 7. Difference Between Iteration and Recursion

Feature	Recursion	Iteration
Approach	Function calls itself	Uses loops (for/while)
Stopping Point	Base case	Loop condition
Memory Usage	Uses stack memory	Less memory usage
Performance	Can be slower	Often faster
Code Simplicity	Simpler for some problems	Simple for linear tasks

🌀 Disadvantages of Recursion

- More memory usage (function call stack)
- Slower due to overhead of multiple function calls
- Risk of stack overflow if base case is missing

6. Passing Arrays to Functions (1 Hour)

In C programming, **arrays** are passed to functions **by reference**, meaning that the function receives a reference (or pointer) to the original array rather than a copy of it. As a result, any modifications made to the array inside the function directly affect the original array in the calling function. This behavior makes arrays unique when passed to functions compared to basic data types, which are typically passed by value.

◆ What Does "Passing by Reference" Mean?

When passing an array to a function, you are passing the **address of the first element** of the array, which is known as **passing by reference**. This means that the function operates directly on

the original array, rather than a copy of it. As a result, changes made to the array inside the function will reflect in the original array.

Why is this important?

- **Efficiency:** Passing large arrays by reference is more efficient than passing by value because no new copy of the array needs to be created. Instead, only the array's address is passed.
- **Data Modification:** If the function is intended to modify the array's contents (e.g., sorting an array or updating elements), it can directly affect the original array.

◆ Syntax for Passing Arrays to Functions

When you pass an array to a function, you pass only the **address of the first element**. In the function, this is treated as a pointer to the array.

Function Definition Syntax:

```
c
void functionName(int arr[], int size) {
    // Function body
}
```

- **arr[]:** The array parameter, which is treated as a pointer to the first element of the array.
- **size:** The number of elements in the array (optional but often passed for size-related operations).

Function Call Syntax:

```
c
functionName(arr, size);
```

- **arr:** The actual array being passed.
- **size:** The size of the array (number of elements) to help the function process the array correctly.

◆ Example: Passing Arrays to Functions

Let's take a look at a basic example of passing an array to a function and printing its elements.

Example 1: Printing Array Elements

c

```
#include <stdio.h>

// Function to print all elements of the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]); // Print each element
    }
    printf("\n");
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5}; // Declare an array
    int size = sizeof(numbers) / sizeof(numbers[0]); // Calculate array size

    // Pass the array and its size to the function
    printArray(numbers, size);

    return 0;
}
```

Explanation:

- The **printArray()** function takes two parameters:
 - **arr[]**: The array to be passed by reference.
 - **size**: The number of elements in the array.
- Inside the function, a loop is used to print each element of the array.
- The **sizeof()** operator is used to calculate the size of the array.

Output:

1 2 3 4 5

9. Macros in C (1 Hour)

In C programming, **macros** are a powerful tool provided by the **C preprocessor** to facilitate code reuse, improve performance, and define constants or expressions. Macros allow you to define snippets of code that can be reused throughout your program. Since macros are handled by the preprocessor, they can be substituted in place of the macro name before the actual compilation begins, offering performance improvements.

◆ What is a Macro?

A **macro** is a **preprocessor directive** that allows you to define a constant value or an expression that can be substituted into your code wherever the macro is used. Macros are often used for:

- **Defining constants** that remain unchanged throughout the program.
- **Code reuse:** Repeating blocks of code with a single definition.
- **Improving performance:** By avoiding function calls and using inline expressions.

Basic Syntax of a Macro:

C

```
#define MACRO_NAME value
```

- **#define** is a preprocessor directive used to define the macro.
- **MACRO_NAME** is the name of the macro.
- **value** is the value or expression associated with the macro.

Macros are expanded **before the program is compiled** by the preprocessor, meaning they are simply text replacements that are inserted into the code.

◆ Types of Macros

1. Object-like Macros:

- These are simple constants or fixed values that are used throughout the program.

Example:

C

```
#define PI 3.1415 // Defining a constant for Pi
```

Usage:

C

```
double area = PI * r * r; // PI will be replaced with 3.1415 during preprocessing
```

2. Function-like Macros:

- These macros are defined to perform a specific operation, much like a function, but they do not involve function call overhead.
- Function-like macros can accept arguments and use them in expressions.

Example:

C

```
#define SQUARE(x) ((x)*(x)) // Macro to calculate the square of a
number
```

Usage:

```
c

int result = SQUARE(5); // SQUARE(5) will be replaced with (5)*(5)
```

Here, **SQUARE(x)** is a function-like macro that calculates the square of `x` by substituting the expression `((x)*(x))`.

◆ How Macros Work

- **Preprocessing:** The **C preprocessor** performs macro substitution before the actual compilation starts. This means that any occurrence of the macro name in the code will be replaced with the defined value or expression.
- **No Function Call Overhead:** Since macros are directly replaced in the code, there is no function call overhead, making macros faster than function calls for simple operations.

Example: Basic Macro Usage

```
c

#include <stdio.h>

#define PI 3.1415 // Define a constant for Pi
#define SQUARE(x) ((x)*(x)) // Define a macro to compute the square of a
number

int main() {
    double radius = 5;
    double area = PI * SQUARE(radius); // Using both macros

    printf("Area of circle: %.2f\n", area); // Output: Area of circle: 78.54
    return 0;
}
```

Explanation:

- The **PI** macro is replaced with `3.1415`, and the **SQUARE(radius)** macro is replaced with `((radius)*(radius))`.
- The preprocessor does this substitution before the compilation, so the resulting code would be:

```
c
```

```
double area = 3.1415 * ((radius)*(radius)); // After preprocessing
```

◆ Advantages of Using Macros

1. **Code Reuse:**
 - Macros allow for **code reuse**. Once defined, you can use the macro multiple times throughout the program, reducing redundancy.
2. **Performance:**
 - Since macros are substituted **before compilation**, there is no function call overhead, which can lead to performance improvements in cases of frequently called operations.
3. **Flexibility:**
 - Macros allow you to define **simple constants** (like `#define PI 3.1415`) or **complex expressions** (like `#define SQUARE(x) ((x)*(x))`), which adds flexibility in programming.
4. **Simplify Complex Operations:**
 - Macros can make complex operations (such as mathematical expressions or conditional checks) simpler by abstracting them into a single macro name.

◆ Disadvantages of Using Macros

1. **No Type Checking:**
 - Macros do not perform **type checking**, which means that if the arguments are of the wrong type, there can be unexpected results. For example, if you pass a string to the `SQUARE` macro, it could result in a compile-time error or undefined behavior.
2. **Debugging:**
 - Debugging can be difficult because macros are expanded before the compilation process. This means that errors might appear in the expanded code, making it harder to trace the source of the problem.
3. **Side Effects:**
 - Macros can have **side effects** when arguments are evaluated more than once. For example, in the following macro:

```
c

#define INCREMENT(x) ((x) + 1)
```

Calling `INCREMENT(a++)` can result in undefined behavior because `a++` will be evaluated multiple times in the macro.

◆ Summary of Macros

- **Macros** are **preprocessor directives** used to define constants or expressions that are substituted into the code before compilation.
- **Object-like Macros:** These are used to define constants (e.g., `#define PI 3.1415`).
- **Function-like Macros:** These are used to define reusable code snippets that take arguments (e.g., `#define SQUARE(x) ((x)*(x))`).
- **Advantages:**
 - Code reuse
 - Improved performance (no function call overhead)
 - Simplification of complex logic
- **Disadvantages:**
 - No type checking
 - Difficult to debug
 - Potential side effects from argument evaluation
- **Best Practices:** Always use parentheses, avoid complex logic, and use uppercase for macro names.

10. Storage Classes in C

In C programming, **storage classes** determine the **scope**, **visibility**, and **lifetime** of variables. These aspects control how long a variable exists, where it can be accessed from, and whether it retains its value between function calls.

The **storage class** of a variable specifies the location in memory where the variable is stored, its default value, and how long it persists during the program's execution.

There are **four primary storage classes** in C:

1. **auto**
2. **extern**
3. **static**
4. **register**

Each of these storage classes affects the behavior of variables in different ways, and they can be used in different contexts depending on the scope and lifetime requirements.

◆ 1. auto Storage Class

The **auto** storage class is the **default** storage class for local variables in C. It is used to declare variables that are automatically created when the block in which they are declared is entered and destroyed when the block is exited. **auto** variables exist only within the scope of the function in which they are declared.

Features:

- **Scope:** Local to the function or block.
- **Lifetime:** Exists only during the execution of the function/block. The value is lost once the function ends.
- **Default Value: Garbage** (uninitialized variables will contain random values).

Syntax:

c

```
auto int x; // Declaration of an auto variable (not commonly used, as 'auto'
is default)
```

Example:

c

```
#include <stdio.h>

void test() {
    auto int x = 10; // auto is optional here, as it is the default storage
class
    printf("%d\n", x); // Outputs: 10
}

int main() {
    test();
    return 0;
}
```

In this example, **x** is an auto variable with local scope, meaning it is created when `test()` is called and destroyed when `test()` finishes.

◆ 2. extern Storage Class

The **extern** storage class is used to declare variables that are defined **outside the current file** (in another file or module). It allows functions or variables to be accessible across multiple files. The **extern** variable refers to an **external variable** declared elsewhere.

Features:

- **Scope:** Global (accessible across files).
- **Lifetime:** Exists for the entire duration of the program.
- **Default Value:** 0 (for numerical types).

Syntax:

```
c
extern int x; // Declares a variable 'x' that is defined outside this file
```

Example:

```
c
#include <stdio.h>

extern int count; // Declares 'count' as an external variable

void increment() {
    count++; // Accessing the external variable
}

int main() {
    count = 5; // Defined here
    increment();
    printf("Count is: %d\n", count); // Outputs: 6
    return 0;
}
```

Here, **count** is declared with the **extern** keyword, which means it is defined elsewhere (in this case, in `main()`). This allows the function `increment()` to access and modify it.

Note: The `extern` keyword is not required if the variable is declared in the same file.

◆ 3. static Storage Class

The **static** storage class is used to preserve the **value of a variable** between function calls.

Static variables retain their values across multiple function calls, unlike local variables (which are created and destroyed every time a function is called).

- When used inside a function, **static variables** retain their value between function calls.
- When used outside a function, **static variables** limit the variable's scope to the file, making it invisible to other files (file-level scope).

Features:

- **Scope:**

- If declared inside a function, it is **local** to that function, but retains its value across multiple calls.
- If declared outside a function, it is **global** to the file.
- **Lifetime:** Exists for the lifetime of the program (until the program ends).
- **Default Value:** 0 (for numerical types).

Syntax:

```
c
static int x = 0; // Static variable
```

Example: Static Variable Inside a Function

```
c
#include <stdio.h>

void test() {
    static int count = 0; // Static variable
    count++;
    printf("%d\n", count); // Prints the count value and retains it between
calls
}

int main() {
    test(); // Outputs: 1
    test(); // Outputs: 2
    test(); // Outputs: 3
    return 0;
}
```

Explanation:

- The **static** variable `count` retains its value between calls to the `test()` function. Each time the function is called, `count` is incremented by 1, and the new value is printed.

Output:

```
1
2
3
```

◆ 4. register Storage Class

The **register** storage class is used to declare variables that should be stored in the **CPU registers** rather than RAM for faster access. It is primarily used for variables that are accessed

frequently within loops. However, the **compiler** decides whether to use a register or not, so the **register** keyword is considered a **suggestion** to the compiler, not a command.

Features:

- **Scope:** Local to the function (like `auto`).
- **Lifetime:** Exists during the execution of the function.
- **Default Value: Garbage** (if uninitialized).
- **No Address:** You cannot take the **address** of a `register` variable using the `&` operator, because it's stored in a CPU register, not in memory.

Syntax:

```
c
register int count; // Declare a register variable
```

Example:

```
c
#include <stdio.h>

void test() {
    register int i; // Register variable
    for (i = 0; i < 5; i++) {
        printf("%d ", i); // Loop runs faster with a register variable
    }
    printf("\n");
}

int main() {
    test();
    return 0;
}
```

Explanation:

- **register int i:** This variable is requested to be stored in a CPU register for faster access.

Note: Modern compilers generally optimize variable storage automatically, and the `register` keyword is rarely necessary.

◆ Summary of Storage Classes

Storage Class	Keyword	Scope	Lifetime	Default Value
Automatic	<code>auto</code>	Local (within function)	During function call	Garbage
External	<code>extern</code>	Global (across files)	Entire program	0
Static	<code>static</code>	Local/Global (function or file scope)	Entire program	0
Register	<code>register</code>	Local (within function)	During function call	Garbage